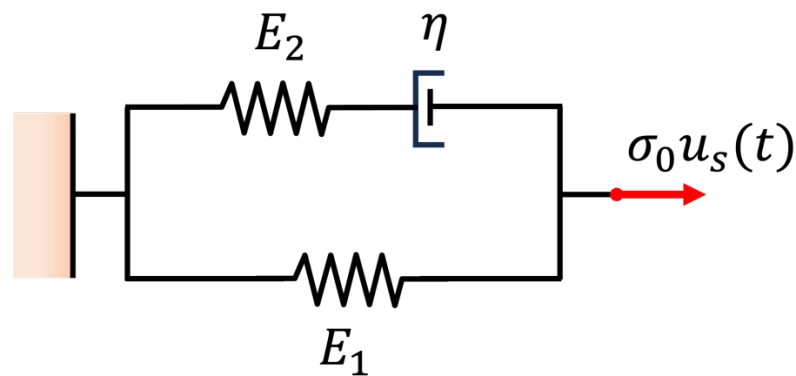


**Due: Friday, 4/4/25 at 11:59pm**

| Problem | Points |
|---------|--------|
| 1       | 16     |
| 2       | 24     |

**Problem #1 – Standard Linear Solid Derivation**

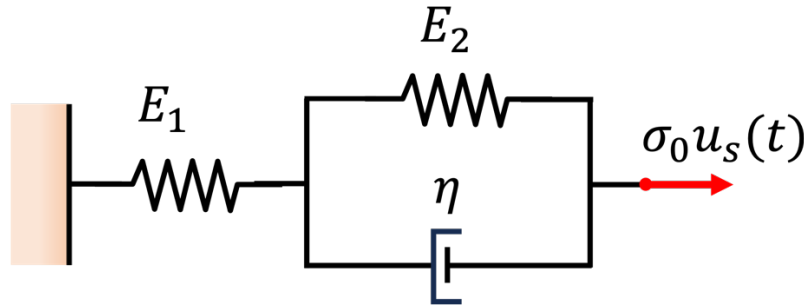
In lecture, we derived the time-dependent strain response of the Kelvin version of the standard linear solid (SLS) viscoelasticity model subjected to a step stress. Derive the time-dependent strain response of the Maxwell representation of the SLS model subjected to a step stress as shown in Figure 1. Show all of your work for full credit. **Hint:** use the same approach that we took in lecture.



**Figure 1:** Maxwell representation of a standard linear solid (SLS) where  $E_1$  and  $E_2$  are moduli values of elastic elements,  $\eta$  is the viscous coefficient, and  $\sigma_0$  is the constant stress step input.

### Problem #2 – Viscoelastic Model Fitting

Let us say you wanted to characterize the viscoelastic properties of an Achilles tendon sample that you obtained from a cadaver. You obtain experimental strain data in response to a step stress input. You hypothesize that you can mathematically describe the tendon response using the standard linear solid (SLS) model below:



**Figure 2:** A schematic of the Kelvin version of the standard linear solid viscoelastic model.

As derived in class, the general form for the differential equation of motion for the model in Figure 2 subjected to a constant stress step is:

$$\dot{\varepsilon}(t) + \frac{E_2}{\eta} \varepsilon(t) = \frac{\sigma_0}{\eta} \left( \frac{E_1 + E_2}{E_1} \right) \quad (1)$$

where  $\varepsilon(t)$  is the time-dependent strain of the material,  $\sigma_0$  is the constant stress input value,  $E_1$  is the modulus in series with the Kelvin component,  $E_2$  is the modulus of the Kelvin component, and  $\eta$  is the viscous term (damping coefficient). If you solve Eq. 1 for strain,  $\varepsilon(t)$ , and also obtain experimental data for strain,  $\varepsilon(t)$ , then you can fit the data with the model and estimate the unknown parameters for the tendon. **Note:** Eq. 1 looks complicated, but there are a lot of constants in there. This is just a first-order differential equation of the form:

$$\dot{\varepsilon}(t) + A\varepsilon(t) = B$$

You already applied a 2 kPa input stress step,  $\sigma_0$ , to the tendon (perhaps using an Instron instrument) and measured displacement vs. time six times ([HW\\_8\\_strain\\_data\\_2025.csv](#)). Now, you need to fit the model response to your experimental response using MATLAB. In class, we determined the solution for the strain response of the Kelvin version of the standard linear solid for an input stress step to be:

$$\varepsilon(t) = -\frac{\sigma_0}{E_2} e^{-t/\tau} + \sigma_0 \left( \frac{E_1 + E_2}{E_1 E_2} \right) \quad (2)$$

where  $\tau$  is the time constant and is equal to  $\frac{\eta}{E_2}$ . This solution can alternatively be written as:

$$\varepsilon(t) = \frac{\sigma_0}{E_1} + \frac{\sigma_0}{E_2} (1 - e^{-t/\tau}) \quad (3)$$

Approach overview:

1. Apply an input stress step,  $\sigma_0 = 2$  kPa
2. Record the strain response,  $\varepsilon(t)$
3. Estimate the unknown parameters ( $E_1$ ,  $E_2$ , and  $\eta$ ) by matching the experimental curves to the theoretical solution of the model (Eq. 2 and 3). You will need to find the time constant,  $\tau$ , as an intermediate step, so report this value as well.
4. Repeat steps 1-3 for all six trials ( $n = 6$  trials)
5. Determine the average and standard deviation of each parameter and the time constant across all six trials and report as: avg.  $\pm$  std. dev. Simply display these values in the Command Window of MATLAB and then publish your script to submit.

**Your tasks:**

2a.) Import the csv data file into MATLAB using the following code:

```
all_Data = importdata('HW_8_strain_data_2025.csv', ',', 1);  
time = all_Data.data(:, 1);  
strain_Data = all_Data.data(:, 2:end);
```

2b.) Plot the experimental strain data for the Achilles tendon sample to get an idea of what you're working with:

```
numDataSets = size(strain_Data, 2);  
  
origData = figure;  
  
for i = 1:numDataSets  
  
    plot(time, strain_Data(:, i), 'ok', 'MarkerSize', 10, 'MarkerFaceColor', [ 1-(i/numDataSets*0.7)  
0 0]);  
    hold on  
  
end
```

2c.) Examine the shape of the curves and imagine what these responses would look like physically. Relate the quantitative features (e.g. y-intercept, slope, steady-state value) of your theoretical plots to the terms in Eq. 2 and Eq. 3 to estimate the unknown parameters of interest ( $E_1$ ,  $E_2$ , and  $\eta$ ) from your model. This is one way you can estimate the material properties of a biomaterial from experimental data. For example:

- What term represents the y-intercept in Eq. 3?
- What is the y-intercept value of one of your theoretical regressions?
- What equation can you now write to relate any of the unknown parameters ( $E_1$ ,  $E_2$ , and  $\eta$ ) to the y-intercept value?
- You have three unknown parameters, so you need two more equations using the 1.) steady-state value and 2.) slope of the linearized form of Eq. 2 (linearize by taking the natural log).

I will now guide you on how to extract the y-intercept, steady-state, and linearized slope values from experimental data using MATLAB.

- 2d.) The y-intercept can be estimated by the y-intercept of a linear regression through the first few points (the code below does the first five points):

```
nf = 5;
for j = 1:numDataSets
    fitVals(j,:) = polyfit(time(1:nf),strain_Data(1:nf,j),1);
    yintVals(j) = fitVals(j,2);
end
param1 = yintVals.*[conversion equation];

param1Avg = mean(param1);
param1StdDev = std(param1);

fprintf('Param1: %.2f \x00B1 %.2f Units\n',
    param1Avg,param1StdDev);
```

Replace “param1” with the name of the parameter. Be careful though; the y-intercept is not a parameter value, but it can be plugged into the equation you derived in Step 3 to obtain a parameter.

- 2e.) Next, let us determine the steady-state value at static equilibrium value. It looks like the strain does reach steady-state, so let us just average the last 10 values of strain for each trial to estimate the steady-state values (each trial will have a slightly different steady-state value due to experimental variation):

```
nf = 10;
for k = 1:numDataSets
    ssVal(k) = mean(strain_Data(end-nf:end,k));
end

param2 = ssVal.*[conversion equation];

param2Avg = mean(param2);
param2StdDev = std(param2);

fprintf('Param2: %.2f \x00B1 %.2f Units\n',
    param1Avg,param1StdDev);
```

Replace “param2” with the name of the parameter.

- 2f.) Last, let us determine the power in the exponential (which is related to  $\tau$ , and  $\tau = \frac{\eta}{E_2}$ ). This value is very easy to obtain once you linearize the exponential data using the

natural log. Our first-order non-homogeneous differential equation in Eq. 1 has the solution shown in Eq. 3, which has the general form:

$$y = A + B(1 - e^{-t/\tau}) \quad (4)$$

can be re-arranged as:

$$A + B - y = Be^{-t/\tau} \quad (5)$$

Linearize this equation by taking the natural logarithm of both sides. You should obtain:

$$\ln(A + B - y) = \ln(Be^{-t/\tau}) \quad (6)$$

which simplifies to:

$$\ln(A + B - y) = -\frac{1}{\tau}t + \ln(B) \quad (7)$$

Transform the experimental data to match Eq. 7 and plot vs. time (**Hint:** compare Eq. 4 to Eq. 3). This will lead to some minor complex values just because some of the values are slightly larger (due to noise) than the average steady state value, but we can just multiply by the complex conjugate to get rid of those warnings. Now, we need to fit a linear regression to find the slope. I will show you how to get the slope but, you have to determine how the slope is related to the parameters in Eq. 3 and Eq. 7. You need to plot six linear regressions and store the slopes. The data gets noisy around steady-state, so I would only use the first 20 or so data points:

```
A = sigma./E1;
B = sigma./E2;
nf = 20; % Ignore the noisy steady-state values in fit
for l = 1:numDataSets
    lin_strain_Data(:,l) = log(A(l)+B(l)-strain_Data(:,l)); %
    Linearizes data
    lin_strain_Data(:,l) = -sqrt(lin_strain_Data(:,l).*
    conj(lin_strain_Data(:,l)));
    fitVals(l,:) = polyfit(time(1:nf),lin_strain_Data(:,l),1);
    slopes(l) = fitVals(l,1);
end
```

figure

```
for m = 1:numDataSets
```

```

plot(time,lin_strain_Data(:,m),'ok','MarkerSize',10,'M
arkerFaceColor',[ 1-(m/numDataSets*0.7)0 0]);
hold on
yFit = polyval(fitVals(m,:),time);
plot(time,yFit,'r--','LineWidth',3,'Color',[1-
(m/numDataSets*0.7) 1-(m/numDataSets*0.7) 1-
(m/numDataSets*0.7)]);

set(gca,'FontSize',20)
ylabel('ln(Strain)')
xlabel('Time (s)')

```

end

```

param3 = slopes.*[conversion equation];
param3Avg = mean(param3);
param3StdDev = std(param3);

fprintf('param3: %.2f \x00B1 %.2f s\n',tauAvg,tauStdDev);

param4 = param3.*[conversion equation];

param4Avg = mean(param4);
param4StdDev = std(param4);

```

Replace “param3” and “param4” with the names of the parameters you are solving for.  
**Hint:** one of these parameters is the time constant tau.

- 2g.) Finally, let us overlay our SLS model over our data. The only reason you should ever use a line plot is if it represents a continuous model. This is one of those times:

```

t = linspace(0,5*mean(tau))';

SLS_Model = @(t) [add a function here that uses your
parameters and t, refer to Eq. 2] ;

figure(origData)

modeled_Data = SLS_Model(t);

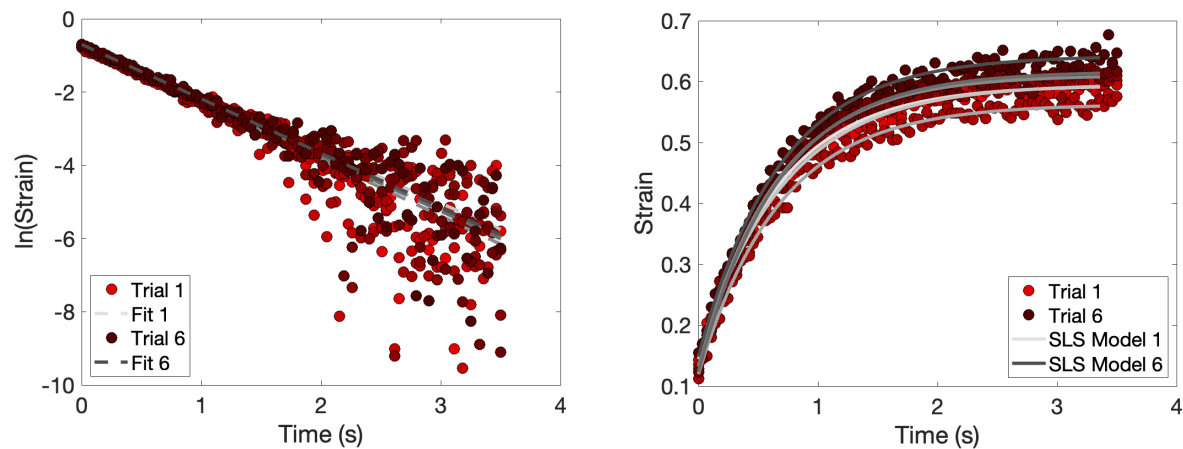
plot(t,SLS_Model(t),'LineWidth',3);
legend('Trial 1',' ',' ',' ',' ','Trial 6','Model
1',' ',' ',' ',' ','Model 6','Location','SouthEast')

```

- 2h.) Publish your final MATLAB script. It should show the final average values and standard deviations (reported as avg.  $\pm$  standard deviation) as well as your plots. FYI – it is better

(and proper) to plot your average and standard deviation instead of all of your trials, but this is OK for this assignment. Submit your publish script to Canvas.

Your plots should look similar to the plots below:



**Submission items:**

1. Submit a pdf containing your SLS derivation and published MATLAB pdf showing your final average values and standard deviations as well as your plots all in a single pdf.  
[yourLastName\\_4211\\_HW\\_8.pdf](#)